



University of Colombo, Sri Lanka



UCSC *University of Colombo School of Computing*

BACHELOR OF SCIENCE IN INFORMATION SYSTEMS

First Year Examination — Semester I— UCSC AY22 [held in Sep\Oct 2025]

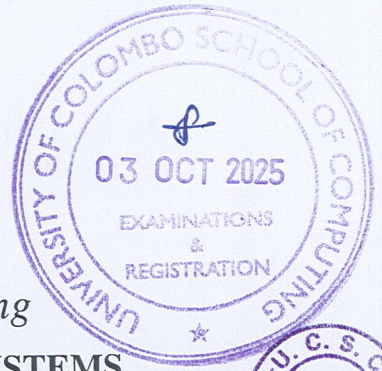
IS1201 — Programming & Problem Solving

(Two (2) Hours)

Answer ALL questions

Number of Pages = 12

Number of Questions = 4



106

To be completed by the candidate

Index Number

--	--	--	--	--	--	--	--

Important Instructions to candidates:

- The medium of instruction and questions is **English**. Students should answer in the medium of English language only.
- Note that questions appear on both sides of the paper. If a page is not printed, please inform the supervisor immediately.
- Write your index number on each and every page of the question paper.
- The duration of the paper is **Two (2)** hours.
- The paper has **4** questions on **12** pages.
- Answer **all** the questions.
- Each question carries exactly **25 marks**.
- **Write your answers on the space provided** on this question paper.
- Any electronic device capable of storing and retrieving text including electronic dictionaries and mobile phones and **Calculators** are **not allowed**.

To be completed by the examiners

1	
2	
3	
4	
Total	

Index Number

--	--	--	--	--	--	--	--

1. (a). Only **ONE** answer is correct for each of the following 10 MCQs.

Select or mark the **best-suited answer** from the given options.

Note: All necessary headers are assumed to be included in the code samples.

[2.5 x 10 = 25 marks]

i.	(A) (B) (C) (D) (E)
ii.	(A) (B) (C) (D) (E)
iii.	(A) (B) (C) (D) (E)
iv.	(A) (B) (C) (D) (E)
v.	(A) (B) (C) (D) (E)

vi.	(A) (B) (C) (D) (E)
vii.	(A) (B) (C) (D) (E)
viii.	(A) (B) (C) (D) (E)
ix.	(A) (B) (C) (D) (E)
x.	(A) (B) (C) (D) (E)

i. Consider the following statements regarding compiled programming languages.

- I. Converted into machine language using an assembler.
- II. Generally faster than interpreted languages.
- III. Java, C, C++ and Python are some examples.

Which of the above statement(s) is/are TRUE?

- | | | |
|---------------------|-------------------|-------------------|
| A. I only. | B. II only. | C. I and II only. |
| D. II and III only. | E. I, II and III. | |

ii. Which of the following is a valid variable name in C?

- | | | |
|-----------|-------------|----------|
| A. my-age | B. 1stPlace | C. _name |
| D. union | E. price\$ | |

iii. If we define `int a=1` and `int b=a--` in a C program, what are the values of `a` and `b` respectively?

- | | | |
|---------|----------|---------|
| A. 1, 0 | B. 1, 1 | C. 0, 1 |
| D. 0, 0 | E. 1, -1 | |

Index Number

--	--	--	--	--	--	--	--

iv. Consider the following statements regarding the loops in C.

- I. `continue` keyword inside a loop will not have any effect.
- II. A `while` loop can always be replaced by a `for` loop.
- III. A `for` loop can always be replaced by a `while` loop.

Which of the above statement(s) is/are TRUE?

- | | | |
|--------------|-------------------|---------------------|
| A. II only. | B. I and II only. | C. II and III only. |
| D. III only. | E. I, II and III. | |

v. What is the output of the following C program?

```
1 void main() {  
2     int xyz=66;  
3     printf("%c, %x", xyz, xyz);  
4 }
```

- | | | |
|-----------|----------|----------|
| A. 66, 66 | B. 66, B | C. B, 42 |
| D. B, 66 | E. B, B | |

vi. If we define `int x=1` and `int y=2` in a C program, what are the values of `x|y` and `x||y` respectively?

- | | | |
|---------|---------|---------|
| A. 1, 2 | B. 2, 1 | C. 1, 3 |
| D. 3, 1 | E. 1, 1 | |

vii. Consider the following statements regarding the functions in C.

- I. A function must return a value.
- II. A function can have default arguments.
- III. Function parameters must be typed.

Which of the above statements is/are TRUE?

- | | | |
|--------------|-------------------|--------------------|
| A. I only. | B. I and II only. | C. I and III only. |
| D. III only. | E. I, II and III. | |

Index Number

--	--	--	--	--	--	--	--

viii. Consider the following piece of code written in C.

```
1  #define f(f1, f2) (f1/f2 + f1 + f2)
2
3  void main() {
4      printf("%.1f", f(5, 2.5));
5  }
```

Which of the following is the correct output of this program?

- | | | |
|--------|-------------------|---------|
| A. 7.5 | B. 10.0 | C. 12.5 |
| D. 9.5 | E. Compiler Error | |

ix. Consider the following code written in C:

```
1  void main() {
2      int a[x][y][z] = {{{1,2},{3,4},{5,6}},
3                          {{7,8},{9,10},{11,12}}};
4  }
```

What would be the correct values of x, y, and z respectively.

- | | | |
|----------|----------|----------|
| A. 3,2,2 | B. 2,2,3 | C. 3,2,3 |
| D. 2,3,2 | E. 2,1,6 | |

x. If we define another array (b) in the above program as

`int b[] = {13,14,15},`

what is the value of `a[2][2][1]+b[1]`?

- | | | |
|-------|-------|-------|
| A. 23 | B. 24 | C. 25 |
| D. 26 | E. 27 | |

Index Number

--	--	--	--	--	--	--	--

2. Write the outputs of the following code snippets written in C.
Assume that all the other required lines of codes are in place to run these programs.

```
(a) 1 while (x>0) {  
    2     x-=1;  
    3     if (x==1) {continue;}  
    4     printf("%d", x);  
    5 }
```

.....

[2 marks]

```
(b) 1 char x = 'A';  
    2 int y = 1;  
    3 printf("%d", sizeof(x+y));
```

.....

[2 marks]

```
(c) 1 int x = 20; int y = 4;  
    2 x %= --y;  
    3 printf("%d", x*y);
```

.....

[2 marks]

```
(d) 1 int x = 2%1;  
    2 char y = (x) ? 'a' : 'b';  
    3 printf("%c", y);
```

.....

[2 marks]

```
(e) 1 int x=3, y=2;  
    2 printf("%d%d", (x&y), (x^y));
```

.....

[2 marks]

Index Number

--	--	--	--	--	--	--	--

```
(f) 1 int a = 1, b = 0, c = a!=b, d = 1, e;
    2 e = (a==c) || (a<=b) && (c>d);
    3 printf("%d", e);
```

[3 marks]

```
(g) 1 int n = x;
    2 switch(n) {
    3     case 3:
    4         printf("A");
    5     case 1:
    6         printf("B");
    7         break;
    8     default:
    9         printf("C"); }
```

when; x=1: x=2: x=3:

[3 marks]

```
(h) 1 int f(int x) {
    2     if (x<=1) return x;
    3     else return x-f(x-1); }
    4 int main() {
    5     printf("%d", f(4)); }
```

[3 marks]

```
(i) 1 #define f1 'A', 66
    2 #define f2(x, y) printf ("%o" #y "%o" #x , f1)
    3 int main() { f2(c,d); }
```

[3 marks]

```
(j) 1 #define A 10
    2 #undef A
    3 int main() {
    4     #if A
    5         printf("%d", A);
    6     #endif
    7     printf("%d", 'B'); }
```

[3 marks]

--	--	--	--	--	--	--

1. The Commission has received a request from the Government of the State of New York for a report on the progress of the work of the Commission since its establishment in 1946.

(b). Consider the following code snippets written in C and state whether the given statements after each code snippet are True (T) or False (F).

1. `p` is a pointer to an integer.
2. `*p = 10;` changes the value of variable `x`.
3. After `p = &y;`, the pointer `p` points to `y`.
4. The output of the `printf` statement is 15.
5. After execution, the value of `x` is still 5.

7

Index Number

--	--	--	--	--	--	--	--

```
ii. 1 int a = 10;
    2 const int b = 20;
    3 const int *p = &b;
    4 p = &a;
    5 printf("%d\n", *p);
```

1. p is a pointer to a constant integer.
2. *p = 30; is a valid assignment that changes the value of a.
3. After p = &a;, is a valid assignment because p can point to another integer.
4. The output of the printf statement is 10.
5. A pointer to a constant means the pointer cannot point to another variable.

[5 marks]

```
iii. 1 int x = 5;
    2 int *p = &x;
    3 int **pp = &p;
    4 **pp = 20;
    5 printf("%d\n", x);
```

1. pp is a pointer to an integer.
2. **pp = 20; changes the value of x.
3. The output of the printf statement is 20.
4. pp stores the address of variable x.
5. If we change p = NULL;, then accessing **pp afterwards would cause undefined behaviour.

[5 marks]

Index Number

--	--	--	--	--	--	--	--

- (c). Briefly explain the difference between the `malloc` and `calloc` functions for dynamic memory allocation in C, using suitable examples.

--

[6 marks]

Index Number

--	--	--	--	--	--	--	--

4. (a). The `deleteElement` function is a `void` function that takes three parameters: an integer array `arr[]`, a pointer to an integer `*n` representing the size of the array, and an integer `pos` that specifies the position of the element to be deleted.

The function removes the element at the specified position by shifting all subsequent elements one position to the left. This overwrites the element that needs to be deleted and updates the array size to reflect the removal.

For example, if you have an integer array of size 4, defined as `int array[] = {10, 20, 30, 40}`, and you want to delete the element 20, the value of `pos` would be 2. After the deletion, the size of the array would be 3.

Complete the function below by filling in the missing parts A – C.

```
void deleteElement(int array[], int *n, int pos){  
  
    for(..... A .....){  
  
        array[i] = ..... B .....;  
    }  
  
    *n = ..... C .....;  
}
```

(A).

(B).

(C).

[5 marks]

- (b). Assume you are tasked with creating a simple banking application using the C programming language. This application should manage the bank accounts of customers, with each account containing the information: account number, account holder's name, and account balance. The application should also support basic banking operations, including creating bank accounts, depositing and withdrawing money, checking account balances, and displaying account details.

Index Number

--	--	--	--	--	--	--	--

- i. Briefly explain why a **structure** is more suitable than a union for storing bank account information.

--

[3 marks]

- ii. Define a global structure to represent a bank account based on the specified requirements.

--

[4 marks]

- iii. Assuming a maximum of 1000 customers, write the C code snippet to create an array capable of holding 1000 bank accounts.

--

[2 marks]

Index Number

--	--	--	--	--	--	--	--

- iv. Write a C function named `searchAccount` that searches for an account by its account number within the array declared in part (iii). The function should return the index of the account if found; otherwise, it should return -1. Assume that the current count of bank accounts is stored in a global variable called `accountCount`.

[8 marks]

- v. Complete the `withdraw` function, which accepts an account number and an amount as parameters, and performs the withdrawal if the balance is sufficient. Fill in the missing parts A-C below.

```
void withdraw(char accountNumber[], double amount) {  
  
    int index = searchAccount(accountNumber);  
  
    if (index == ... A ...) {  
        printf("Account not found!\n");  
        return;  
    }  
  
    if (..... B .....) {  
        ..... C .....;  
        printf("Withdrawal successful!\n");  
    } else {  
        printf("Insufficient balance!\n");  
    }  
}
```

(A).

(B).

(C).

[3 marks]
